

```
import javax.swing.*;
```

```
import java.awt.*;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class Main extends JFrame {
```

```
// -----
```

```
// Booking Model
```

```
// -----
```

```
static class Booking { 4 usages
```

```
    private final String passengerName; 2 usages
```

```
    private final String seatNumber; 2 usages
```

```
    Booking(String passengerName, String seatNumber) { 1 usage
```

```
        this.passengerName = passengerName;
```

```
        this.seatNumber = seatNumber;
```

```
    }
```

```
    public String getPassengerName() { 1 usage
```

```
        return passengerName;
```

```
    }
```

```
    public String getSeatNumber() { no usages
```

```
        return seatNumber;
```

```
    }
```

```
}
```

```
// Centralized seat allocation
```

```
static class BookingManager { 2 usages
```

```
    private final Map<String, Booking> bookings = new HashMap<>(); 6 usages
```

```
    // Seats in the bus/train
```

```
class Main extends JFrame {
static class BookingManager { 2 usages
    private final Map<String, Booking> bookings = new HashMap<>(); 6 usages

    // Seats in the bus/train
    private final String[] seats = { 2 usages
        "A1", "A2", "A3", "A4",
        "B1", "B2", "B3", "B4",
        "C1", "C2", "C3", "C4",
        "D1", "D2", "D3", "D4"
    };

    public boolean isSeatAvailable(String seat) { 6 usages
        return !bookings.containsKey(seat);
    }

    public void bookSeat(String passenger, String seat) 1 usage
        throws IllegalArgumentException {

        if (passenger == null || passenger.trim().isEmpty()) {
            throw new IllegalArgumentException(
                "Passenger name cannot be empty.");
        }

        if (seat == null || !isSeatAvailable(seat)) {
            throw new IllegalArgumentException(
                "The selected seat is not available.");
        }

        bookings.put(
            seat,
            new Booking(passenger.trim(), seat)
        );
    }
}
```



```

6      public class Main extends JFrame {
31          static class BookingManager { 2 usages
47              public void bookSeat(String passenger, String seat) 1 usage
                    "The selected seat is not available.";
57              }
58
59              bookings.put(
60                  seat,
61                  new Booking(passenger.trim(), seat)
62              );
63          };
64      }
65
66      public void cancelBooking(String seat) { 1 usage
67          if (!bookings.containsKey(seat)) {
68              throw new IllegalArgumentException(
69                  "No booking exists for seat " + seat);
70          }
71
72          bookings.remove(seat);
73      }
74
75      public Booking getBooking(String seat) { 1 usage
76          return bookings.get(seat);
77      }
78
79      public String[] getSeats() { 2 usages
80          return seats;
81      }
82
83      public int getAvailableSeatCount() { 1 usage
84          return seats.length - bookings.size();
85      }
86  }
87
88  private final BookingManager bookingManager;

```

```
6 public class Main extends JFrame {
```

```
106 private void createMenu() { 1 usage
```

```
107  
108 JMenuBar menuBar = new JMenuBar();
```

```
109  
110 JMenu bookingMenu = new JMenu( s: "Booking");
```

```
111  
112 JMenuItem bookItem = new JMenuItem( text: "Book Ticket");
```

```
113 JMenuItem cancelItem = new JMenuItem( text: "Cancel Ticket");
```

```
114 JMenuItem exitItem = new JMenuItem( text: "Exit");
```

```
115  
116 bookItem.addActionListener(e -> openBookingWindow());
```

```
117  
118 cancelItem.addActionListener(e -> openCancellationWindow());
```

```
119  
120 exitItem.addActionListener(e -> System.exit( status: 0));
```

```
121  
122 bookingMenu.add(bookItem);
```

```
123 bookingMenu.add(cancelItem);
```

```
124 bookingMenu.addSeparator();
```

```
125 bookingMenu.add(exitItem);
```

```
126  
127 menuBar.add(bookingMenu);
```

```
128  
129 setJMenuBar(menuBar);
```

```
130 }
```

```
131 // -----
```

```
132 // Main Panel
```

```
133 // -----
```

```
134 private void createMainPanel() { 1 usage
```

```
135  
136 JPanel mainPanel = new JPanel(new BorderLayout( hgap: 15, vgap: 15));
```

```
137 mainPanel.setBorder(
```

```
138 BorderFactory.createEmptyBorder( top: 20, left: 20, bottom: 20, right: 20);
```



```
    main extends JFrame {  
private void openBookingWindow() { 2 usages  
};
```

```
bookingDialog.setSize( width: 550, height: 500);  
bookingDialog.setLocationRelativeTo(this);
```

```
JPanel panel = new JPanel(new BorderLayout( hgap: 10, vgap: 10));  
panel.setBorder(  
    BorderFactory.createEmptyBorder( top: 15, left: 15, bottom: 15, right: 15)  
);
```

```
// Passenger information
```

```
JPanel passengerPanel = new JPanel(  
    new FlowLayout(FlowLayout.LEFT)
```

```
);
```

```
JLabel nameLabel = new JLabel( text: "Passenger Name:");  
JTextField nameField = new JTextField( columns: 20);
```

```
passengerPanel.add(nameLabel);  
passengerPanel.add(nameField);
```

```
panel.add(passengerPanel, BorderLayout.NORTH);
```

```
// Seat selection
```

```
JPanel seatPanel = new JPanel(  
    new GridLayout( rows: 4, cols: 4, hgap: 10, vgap: 10)  
);
```

```
JLabel selectedLabel = new JLabel(  
    text: "Selected Seat: None",  
    SwingConstants.CENTER  
);
```

```
final String[] selectedSeat = null;
```



```

class Main extends JFrame {
    private void openBookingWindow() { 2 usages
        new JLabel(
            text: "Selected Seat: None",
            SwingConstants.CENTER
        );

        final String[] selectedSeat = {null};

        Map<String, JButton> seatButtons = new HashMap<>();

        for (String seat : bookingManager.getSeats()) {
            JButton seatButton = new JButton(seat);

            seatButtons.put(seat, seatButton);

            updateSeatButton(seatButton, seat);

            seatButton.addActionListener(e -> {

                String currentSeat = seatButton.getText();

                if (!bookingManager.isSeatAvailable(currentSeat)) {
                    JOptionPane.showMessageDialog(
                        bookingDialog,
                        message: "This seat is already booked.",
                        title: "Seat Unavailable",
                        JOptionPane.WARNING_MESSAGE
                    );
                    return;
                }
            }
        }
    }
}

```



```
button.addActionListener(e -> {
```

```
    selectedSeat[0] = currentSeat;
```

```
    selectedLabel.setText(  
        "Selected Seat: " + currentSeat  
    );
```

```
    // Reset colors of available seats
```

```
    for (Map.Entry<String, JButton> entry :  
        seatButtons.entrySet()) {
```

```
        String seatName = entry.getKey();
```

```
        JButton button = entry.getValue();
```

```
        if (bookingManager.isSeatAvailable(seatName)) {  
            button.setBackground(null);
```

```
        }
```

```
    }
```

```
    // Highlight selected seat
```

```
    seatButton.setBackground(Color.GREEN);
```

```
});
```

```
seatPanel.add(seatButton);
```

```
panel.add(seatPanel, BorderLayout.CENTER);
```

```
// Bottom controls
```

```
JPanel bottomPanel = new JPanel(  
    new BorderLayout(10, 10)
```



```
public class Main extends JFrame {  
    private void openBookingWindow() { 2 usages  
        panel.add(seatPanel, BorderLayout.CENTER);  
  
        // Bottom controls  
        JPanel bottomPanel = new JPanel(  
            new BorderLayout( hgap: 10, vgap: 10)  
        );  
  
        bottomPanel.add(  
            selectedLabel,  
            BorderLayout.NORTH  
        );  
  
        JPanel actionPanel = new JPanel();  
        JButton confirmButton = new JButton( text: "Confirm Booking");  
        JButton closeButton = new JButton( text: "Close");  
  
        confirmButton.addActionListener(e -> {  
            try {  
                String passenger = nameField.getText().trim();  
  
                if (passenger.isEmpty()) {  
                    throw new IllegalArgumentException(  
                        "Please enter passenger name."  
                    );  
                }  
  
                if (selectedSeat[0] == null) {  
                    throw new IllegalArgumentException(  
                        "Please select a seat."  
                    );  
                }  
            }  
        })  
    }  
}
```

```
showConfirmationWindow(  
    bookingDialog,  
    passenger,  
    selectedSeat[0]  
);
```

```
statusLabel.setText(  
    "Ticket booked: " +  
        passenger +  
        " - Seat " +  
        selectedSeat[0]  
);
```

```
bookingDialog.dispose();
```

```
} catch (IllegalArgumentException ex) {
```

```
    JOptionPane.showMessageDialog(  
        bookingDialog,  
        ex.getMessage(),  
        title: "Booking Error",  
        JOptionPane.ERROR_MESSAGE  
    );
```

```
} catch (Exception ex) {
```

```
    JOptionPane.showMessageDialog(  
        bookingDialog,  
        message: "Unexpected error: " + ex.getMessage(),  
        title: "System Error",  
        JOptionPane.ERROR_MESSAGE  
    );
```



```

public class Main extends JFrame {
    private void openBookingWindow() { 2 u
        confirmButton.addActionListener(e -

            } catch (Exception ex) {

                JOptionPane.showMessageDialog(
                    bookingDialog,
                    message: "Unexpected error",
                    title: "System Error",
                    JOptionPane.ERROR_MESSAGE
                );
            }
        });

        closeButton.addActionListener(
            e -> bookingDialog.dispose()
        );

        actionPanel.add(confirmButton);
        actionPanel.add(closeButton);

        bottomPanel.add(actionPanel, BorderLayout.SOUTH);

        panel.add(bottomPanel, BorderLayout.SOUTH);

        bookingDialog.add(panel);
        bookingDialog.setVisible(true);
    }

    private void updateSeatButton( 1 usage
        JButton button,
        String seat
    ) {
        if (bookingManager.isSeatAvailable(seat)) {

```



```
void showConfirmationDialog() {
    dialog.ModalityType.APPLICATION_MODAL

};

confirmationDialog.setSize( width: 400, height: 300);
confirmationDialog.setLocationRelativeTo(parent);

JPanel panel = new JPanel(
    new BorderLayout( hgap: 10, vgap: 10)
);

panel.setBorder(
    BorderFactory.createEmptyBorder(
        top: 20, left: 20, bottom: 20, right: 20
    )
);

JLabel title = new JLabel(
    text: "Booking Confirmed!",
    SwingConstants.CENTER
);

title.setFont(
    new Font( name: "Arial", Font.BOLD, size: 22)
);

JPanel details = new JPanel(
    new GridLayout( rows: 3, cols: 1, hgap: 5, vgap: 5)
);

details.add(
    new JLabel(
        text: "Passenger: " + passenger,
        SwingConstants.CENTER
    )
);
```



```
ivate void openCancellationWindow()
```

```
);
```

```
cancelDialog.setSize( width: 400, height: 250);  
cancelDialog.setLocationRelativeTo(this);
```

```
JPanel panel = new JPanel(  
    new BorderLayout( hgap: 10, vgap: 10)  
);
```

```
panel.setBorder(  
    BorderFactory.createEmptyBorder(  
        top: 20, left: 20, bottom: 20, right: 20  
    )  
);
```

```
JLabel label = new JLabel(  
    text: "Enter Seat Number:",  
    SwingConstants.CENTER  
);
```

```
JTextField seatField = new JTextField();
```

```
JPanel inputPanel = new JPanel(  
    new BorderLayout( hgap: 5, vgap: 5)  
);
```

```
inputPanel.add(label, BorderLayout.NORTH);  
inputPanel.add(seatField, BorderLayout.CENTER);
```

```
JPanel buttonPanel = new JPanel();
```

```
JButton cancelButton = new JButton(
```



```
cancelButton.addActionListener(e -> {
```

```
    try {
```

```
        String seat = seatField.getText()  
            .trim()  
            .toUpperCase();
```

```
        if (seat.isEmpty()) {  
            throw new IllegalArgumentException(  
                "Enter a seat number."  
            );  
        }
```

```
        Booking booking =  
            bookingManager.getBooking(seat);
```

```
        if (booking == null) {  
            throw new IllegalArgumentException(  
                "No booking exists for seat "  
                + seat  
            );  
        }
```

```
        int answer = JOptionPane.showConfirmDialog(  
            cancelDialog,  
            message: "Cancel booking for "  
                + booking.getPassengerName()  
                + " on seat "  
                + seat + "?",  
            title: "Confirm Cancellation",  
            JOptionPane.YES_NO_OPTION
```

```
    );
```



```
public class Main extends JFrame {  
    private void openCancellationWindow() { 2 usages  
        cancelButton.addActionListener(e -> {  
            JOptionPane.showMessageDialog(  
                cancelDialog,  
                ex.getMessage(),  
                title: "Cancellation Error",  
                JOptionPane.ERROR_MESSAGE  
            );  
        }  
    );  
};
```

```
        closeButton.addActionListener(  
            e -> cancelDialog.dispose()  
        );
```

```
        buttonPanel.add(cancelButton);  
        buttonPanel.add(closeButton);
```

```
        panel.add(inputPanel, BorderLayout.CENTER);  
        panel.add(buttonPanel, BorderLayout.SOUTH);
```

```
        cancelDialog.add(panel);  
        cancelDialog.setVisible(true);  
    }
```

```
// -----  
// Availability  
// -----
```

```
private void showAvailability() { 1 usage
```

```
    StringBuilder available = new StringBuilder();
```

```
    for (String seat : bookingManager
```



```
        "Seat Availability",  
        JOptionPane.INFORMATION_MESSAGE  
    );  
}
```

```
// -----  
// Main Method  
// -----
```

```
public static void main(String[] args) {
```

```
    // Ensures Swing components are created  
    // on the Event Dispatch Thread.
```

```
    SwingUtilities.invokeLater(() -> {
```

```
        Main app = new Main();
```

```
        app.setVisible(true);
```

```
    });  
}
```




```
public class Main extends JFrame {  
    private void showAvailability() { 1 usage
```

```
JOptionPane.showMessageDialog(  
    parentComponent: this,
```

```
    message: "Available seats:\n\n"
```

14 3 ^ v

Booking

Ticket Booking System

Ticket Booking System

Book Ticket

Cancel Ticket

View Availability

Exit

Welcome! Choose an operation.

634



```
Main app = new Main();
```

```
app.setVisible(true);
```

```
});
```

```
}
```

```
}
```

Book Ticket

Passenger Name:

A1

Available

A3

A4

B1

B2

B3

B4

C1

C2

C3

C4

D1

D2

D3

D4

Selected Seat: None

Confirm Booking

Close


```
1 import java.applet.Applet;
2 import java.awt.*;
3 import java.awt.event.*;
4
5 public class Main extends Applet
6     implements ActionListener {
7
8     TextField nameField; 6 usages
9     TextField emailField; 6 usages
10    TextArea commentArea; 5 usages
11
12    CheckboxGroup ratingGroup; 7 usages
13    Checkbox excellent; 2 usages
14    Checkbox good; 2 usages
15    Checkbox average; 2 usages
16    Checkbox poor; 2 usages
17
18    Button submitButton; 4 usages
19    Button resetButton; 4 usages
20
21    Label resultLabel; 9 usages
22    TextArea resultArea; 6 usages
23
24    @Override
25    public void init() {
26
27        setLayout(new BorderLayout( hgap: 10, vgap: 10));
28        setBackground(new Color( r: 240, g: 245, b: 250));
29
30        Label title = new Label(
31            text: "Customer Feedback Form",
32            Label.CENTER
33        );
34
35        title.setFont(
```


Button submitButton; 4 usages

Button resetButton; 4 usages

Label resultLabel; 9 usages

TextArea resultArea; 6 usages

@Override

public void init() {

setLayout(new BorderLayout(hgap: 10, vgap: 10));

setBackground(new Color(r: 240, g: 245, b: 250));

Label title = new Label(

text: "Customer Feedback Form",

Label.CENTER

);

title.setFont(

new Font(name: "Arial", Font.BOLD, size: 22)

);

add(title, BorderLayout.NORTH);

// Form panel

Panel formPanel = new Panel(

new GridLayout(rows: 5, cols: 2, hgap: 10, vgap: 10

);

formPanel.add(new Label(text: "Customer Name:"));

nameField = new TextField();

formPanel.add(nameField);


```
public void  
add(title, BorderLayout.NORTH);  
  
// Form panel  
Panel formPanel = new Panel(  
    new GridLayout(5, 2, 10, 10)  
);  
  
formPanel.add(new Label(text: "Customer Name:"));  
  
nameField = new TextField();  
formPanel.add(nameField);  
  
formPanel.add(new Label(text: "Email:"));  
  
emailField = new TextField();  
formPanel.add(emailField);  
  
formPanel.add(new Label(text: "Overall Rating:"));  
  
Panel ratingPanel = new Panel(  
    new FlowLayout(FlowLayout.LEFT)  
);  
  
ratingGroup = new CheckboxGroup();  
  
excellent = new Checkbox(  
    label: "Excellent", ratingGroup, state: false  
);  
  
good = new Checkbox(  
    label: "Good", ratingGroup, state: false  
);
```



```
public class Main extends Applet
{
    public void init() {
        Panel ratingPanel = new Panel(
            new FlowLayout(FlowLayout.LEFT)
        );

        ratingGroup = new CheckboxGroup();

        excellent = new Checkbox(
            label: "Excellent", ratingGroup, state: false
        );

        good = new Checkbox(
            label: "Good", ratingGroup, state: false
        );

        average = new Checkbox(
            label: "Average", ratingGroup, state: false
        );

        poor = new Checkbox(
            label: "Poor", ratingGroup, state: false
        );

        ratingPanel.add(excellent);
        ratingPanel.add(good);
        ratingPanel.add(average);
        ratingPanel.add(poor);

        formPanel.add(ratingPanel);

        formPanel.add(new Label(text: "Comments:"));

        commentArea = new TextArea(
            text: "",
            rows: 4
        );
    }
}
```


main extends Applet

```
public void init() {  
    poor = new Checkbox(  
        label: "Poor", ratingGroup, state: false  
    );
```

```
    ratingPanel.add(excellent);  
    ratingPanel.add(good);  
    ratingPanel.add(average);  
    ratingPanel.add(poor);
```

```
    formPanel.add(ratingPanel);
```

```
    formPanel.add(new Label( text: "Comments:"));
```

```
    commentArea = new TextArea(  
        text: "",  
        rows: 4,  
        columns: 30,  
        TextArea.SCROLLBARS_VERTICAL_ONLY  
    );
```

```
    formPanel.add(commentArea);
```

```
    formPanel.add(new Label( text: "Actions:"));
```

```
    Panel buttonPanel = new Panel(  
        new FlowLayout(FlowLayout.LEFT)  
    );
```

```
    submitButton = new Button( label: "Submit Feedback");  
    resetButton = new Button( label: "Reset");
```

```
    submitButton.addActionListener( this);  
    resetButton.addActionListener( this);
```



```
submitButton = new Button( label: "Submit Feedback");  
resetButton = new Button( label: "Reset");
```

```
submitButton.addActionListener( ! this);  
resetButton.addActionListener( ! this);
```

```
buttonPanel.add(submitButton);  
buttonPanel.add(resetButton);
```

```
formPanel.add(buttonPanel);
```

```
add(formPanel, BorderLayout.CENTER);
```

```
// Result panel
```

```
Panel resultPanel = new Panel(  
    new BorderLayout( hgap: 5, vgap: 5)  
);
```

```
resultLabel = new Label(  
    text: "Feedback Results",  
    Label.CENTER  
);
```

```
resultLabel.setFont(  
    new Font( name: "Arial", Font.BOLD, size: 16)  
);
```

```
resultArea = new TextArea(  
    text: "No feedback submitted yet.",  
    rows: 8,  
    columns: 50,  
    TextArea.SCROLLBARS_VERTICAL_ONLY
```



```
public void init() {
```

```
    resultLabel.setFont(  
        new Font( name: "Arial", Font.BOLD, size: 16)  
    );
```

```
    resultArea = new TextArea(  
        text: "No feedback submitted yet.",  
        rows: 8,  
        columns: 50,  
        TextArea.SCROLLBARS_VERTICAL_ONLY  
    );
```

```
    resultArea.setEditable(false);
```

```
    resultPanel.add(  
        resultLabel,  
        BorderLayout.NORTH  
    );
```

```
    resultPanel.add(  
        resultArea,  
        BorderLayout.CENTER  
    );
```

```
    add(resultPanel, BorderLayout.SOUTH);
```

```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
    if (e.getSource() == submitButton) {  
        submitFeedback();
```

```
    }
```



```
}
```

```
@Override
```

```
public void actionPerformed(ActionEvent e) {
```

```
    if (e.getSource() == submitButton) {
```

```
        submitFeedback();
```

```
    }
```

```
    else if (e.getSource() == resetButton) {
```

```
        resetForm();
```

```
    }
```

```
}
```

```
private void submitFeedback() { 1 usage
```

```
I
```

```
    String name = nameField.getText().trim();
```

```
    String email = emailField.getText().trim();
```

```
    String comments = commentArea.getText().trim();
```

```
    Checkbox selectedRating =
```

```
        ratingGroup.getSelectedCheckbox();
```

```
    // Validation
```

```
    if (name.isEmpty()) {
```

```
        showMessage("Please enter your name.");
```

```
        nameField.requestFocus();
```

```
        return;
```

```
    }
```

```
    if (email.isEmpty()) {
```

```
        showMessage("Please enter your email.");
```

```
        emailField.requestFocus();
```

```
        return;
```



```
class Main extends Applet  
private void submitFeedback() { 1 usage  
    return;  
}
```

```
if (email.isEmpty()) {  
    showMessage("Please enter your email.");  
    emailField.requestFocus();  
    return;  
}
```

```
if (!email.matches(  
    regex: "[A-Za-z0-9+_.-]+@[A-Za-z0-9.-]+.$") {  
  
    showMessage("Please enter a valid email.");  
    emailField.requestFocus();  
    return;  
}
```

```
if (selectedRating == null) {  
    showMessage("Please select a rating.");  
    return;  
}
```

```
if (comments.isEmpty()) {  
    showMessage("Please enter your comments.");  
    commentArea.requestFocus();  
    return;  
}
```

```
String rating =  
    selectedRating.getLabel();
```

```
String result =  
    "Feedback Submitted Successfully!\n\n"
```



```
resultArea.setText(result);
```

```
resultLabel.setText(  
    "Feedback Received - Thank You!"  
);
```

```
resultLabel.setForeground(  
    new Color(r: 0, g: 120, b: 0)  
);
```

```
}
```

```
private void resetForm() { 1 usage
```

```
    nameField.setText("");  
    emailField.setText("");  
    commentArea.setText("");
```

```
    ratingGroup.setSelectedCheckbox(null);
```

```
    resultArea.setText(  
        "No feedback submitted yet."  
    );
```

```
    resultLabel.setText(  
        "Feedback Results"  
    );
```

```
    resultLabel.setForeground(Color.BLACK);
```

```
    nameField.requestFocus();
```

```
}
```



```
public static void main(String[] args) {
```

```
    JFrame frame = new JFrame(  
        title: "Customer Feedback System"
```

```
    );
```

```
    Main applet = new Main();
```

```
    frame.add(applet);
```

```
    frame.setSize( width: 750, height: 650);
```

```
    frame.setLocationRelativeTo(null);
```

```
    frame.addWindowListener(  
        new WindowAdapter() {
```

```
            @Override
```

```
            public void windowClosing(  
                WindowEvent e) {
```

```
                applet.stop();
```

```
                applet.destroy();
```

```
                frame.dispose();
```

```
                System.exit( status: 0);
```

```
            }
```

```
        }
```

```
    );
```

```
    applet.init();
```

```
    applet.start();
```

```
    frame.setVisible(true);
```

```
}
```


Customer Feedback Form

Customer Name:

Email:

Overall Rating:

☐ Excellent ☐ Good ☐ Average ☐ Poor

Comments:

Actions:

Submit Feedback

Reset

Feedback Results

No feedback submitted yet.